Context switch & Operations on Processes



By:

Dr. H. K. Azad

Assistant Professor (Sr.)

SCOPE

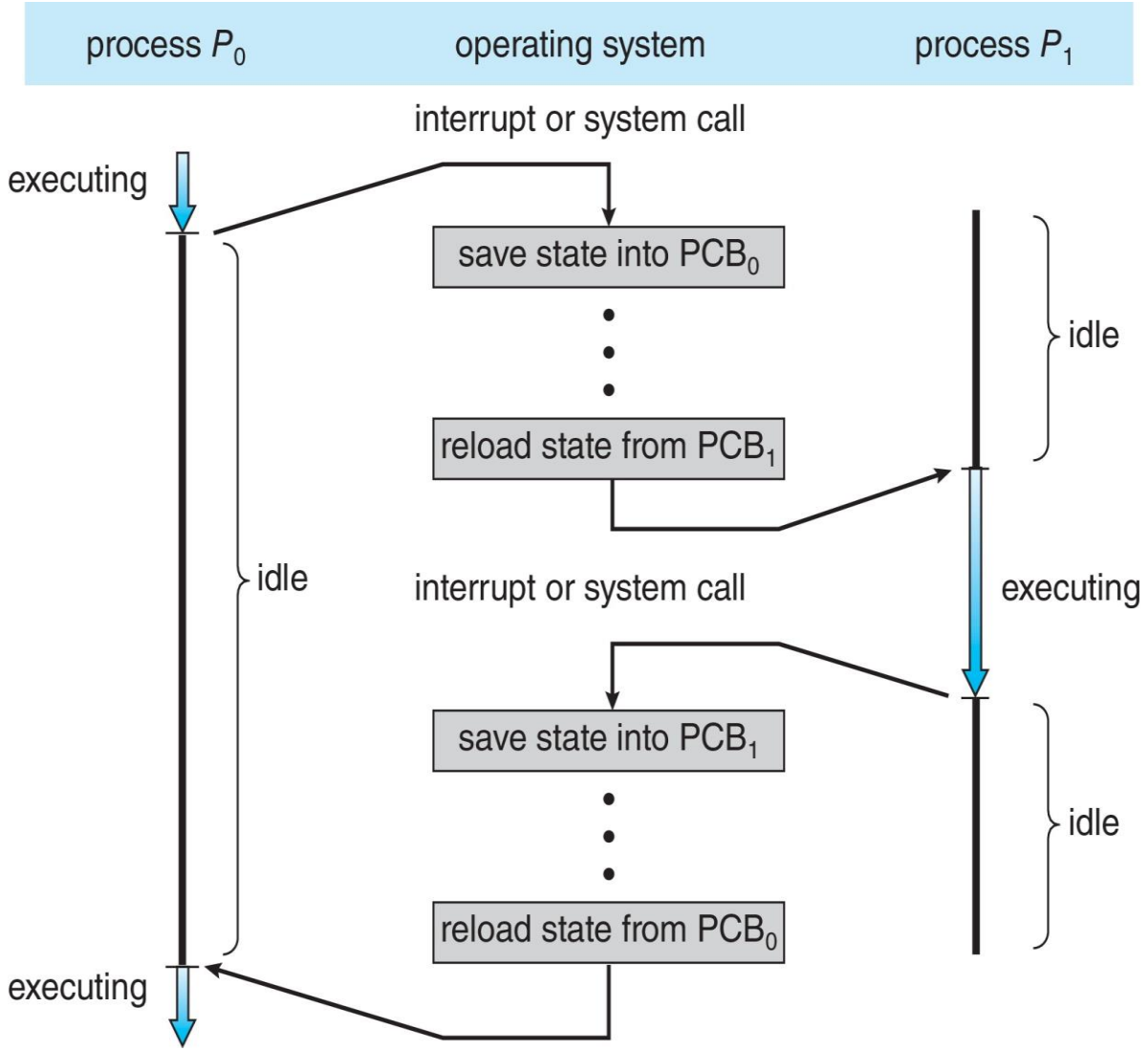
Email: hiteshwarkumar.azad@vit.ac.in

Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is pure overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once

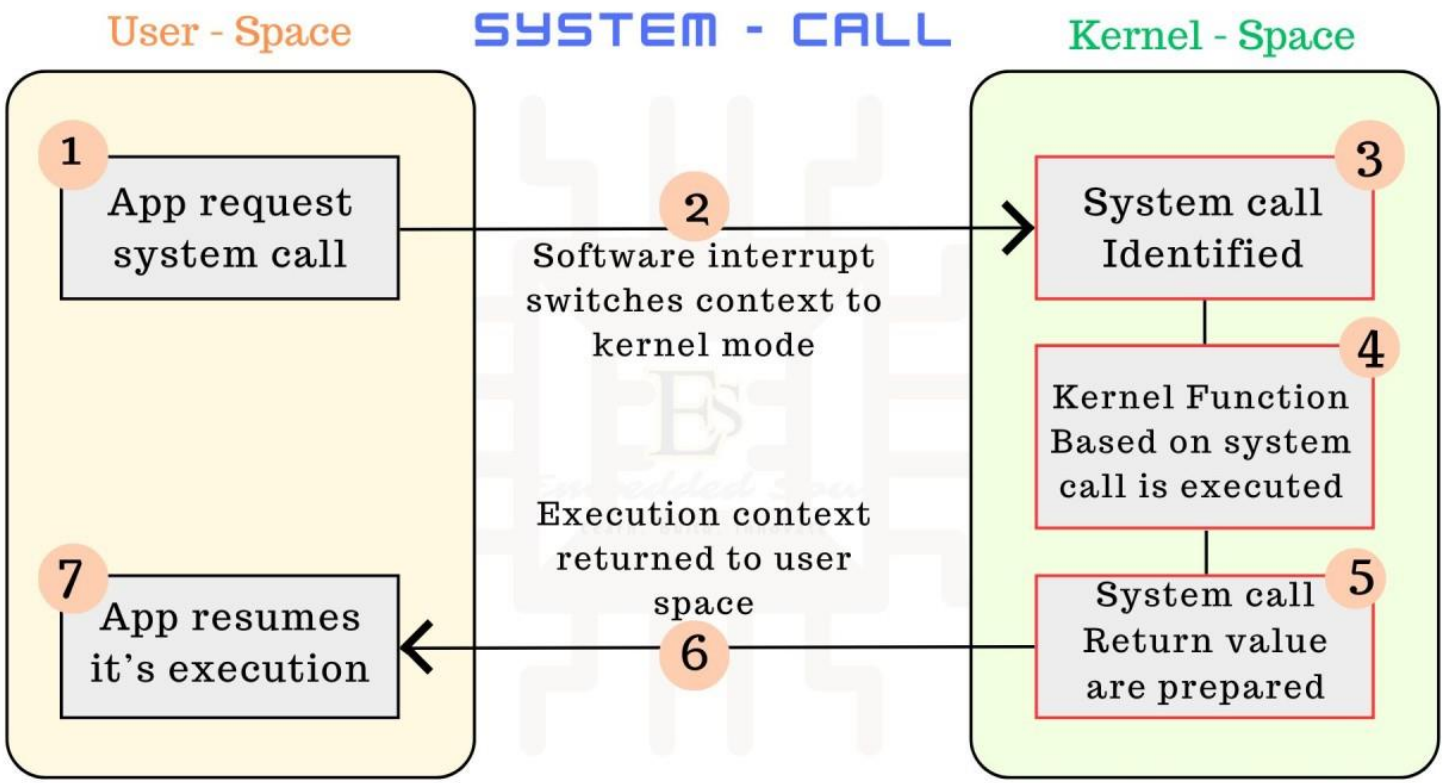
CPU Switch From Process to Process

A context switch occurs when the CPU switches from one process to another.



System Call

- A system call is a programmatic way in which a computer program requests a service from the kernel of the OS.
- Acts as an interface between user programs and the OS.
- Needed for access to hardware (I/O), file systems, process control, etc.



Need for System Calls

Why System Calls Are Needed:

- **User programs** cannot directly access hardware (e.g., disk, memory).
- OS provides a **secure and controlled mechanism** via system calls.
- Ensures **abstraction**, **protection**, and **efficient resource usage**.

- **Example:**
  To read a file:

```
read(fd, buffer, size);
```

Here, `read()` is a system call to request data from a file descriptor.

Types of System Calls:

Category	Example System Calls	Description
Process Control	fork(), exec(), exit()	Create or terminate processes
File Management	open(), read(), write(), close()	File operations
Device Management	ioctl(), read(), write()	Control or access hardware devices
Information Maintenance	getpid(), alarm(), sleep()	Get system or process information
Communication	pipe(), shmget(), msgsnd()	Inter-process communication

System Call Life Cycle

How System Calls Work:

1. **User program** calls a library function (e.g., printf()).
2. Function triggers a **system call** (e.g., write()).
3. CPU **switches to kernel mode**.
4. Kernel executes the request (e.g., write to screen).
5. Control **returns to user mode**.

Example: How printf() Works

- `printf()` → libc library → `write()` system call
- Show a simple code and flow diagram

System Call Interface (SCI)

- The SCI is a set of functions provided by the OS that user-space applications can use to interact with the kernel.
- Provides abstraction for hardware and OS complexity.

```c
#include <stdio.h>
int main() {
    printf("Hello, World!");
    return 0;
}
```

System Call vs Function Call

Aspect	System Call	Function Call
Privilege Level	Kernel mode	User mode
Access Hardware	Yes	No
Performance	Slower (due to mode switch)	Faster
Example	write(), fork()	printf(), strlen()

Common System Calls (Linux):

System Call	Purpose
fork()	Create a new process
exec()	Run a new program
wait()	Wait for child process
read()	Read data
write()	Write data
open()	Open a file
close()	Close a file
getpid()	Get process ID

Operations on Processes

- System must provide mechanisms for:
  - Process creation
  - Process termination

✅ 1. Process Creation

- A new process is created using a system call like `fork()` (Unix) or `CreateProcess()` (Windows).
- The new process (child) is typically a **copy of the parent**.
- Can run the same or different program using `exec()` (after `fork()`).

✅ Process Termination

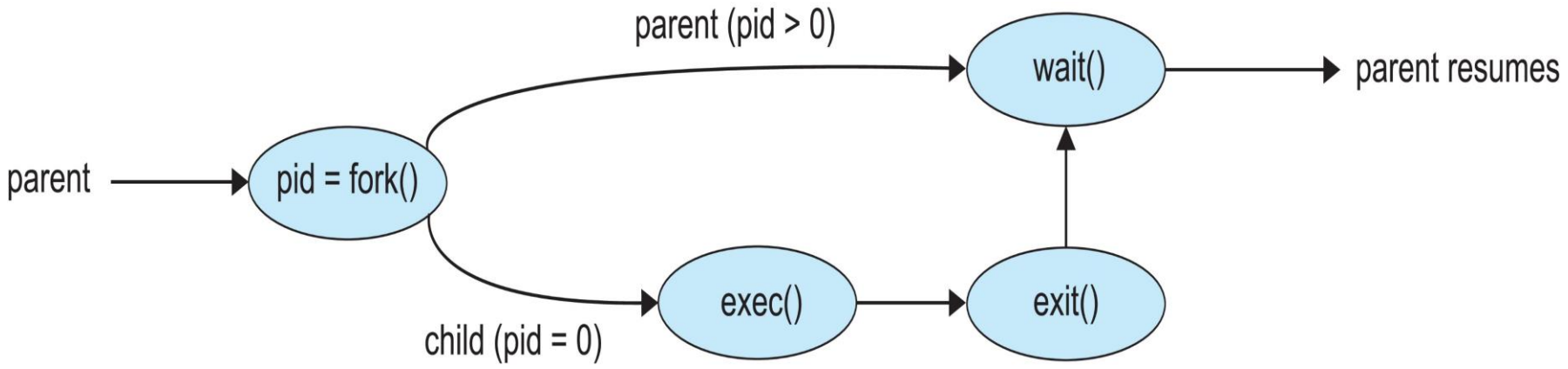
- A process ends when:
  - It completes execution
  - It calls `exit()` or `return`
  - It is killed using `kill()` (Unix) or `TerminateProcess()` (Windows)
- **Resources are released** after termination.

Process Creation

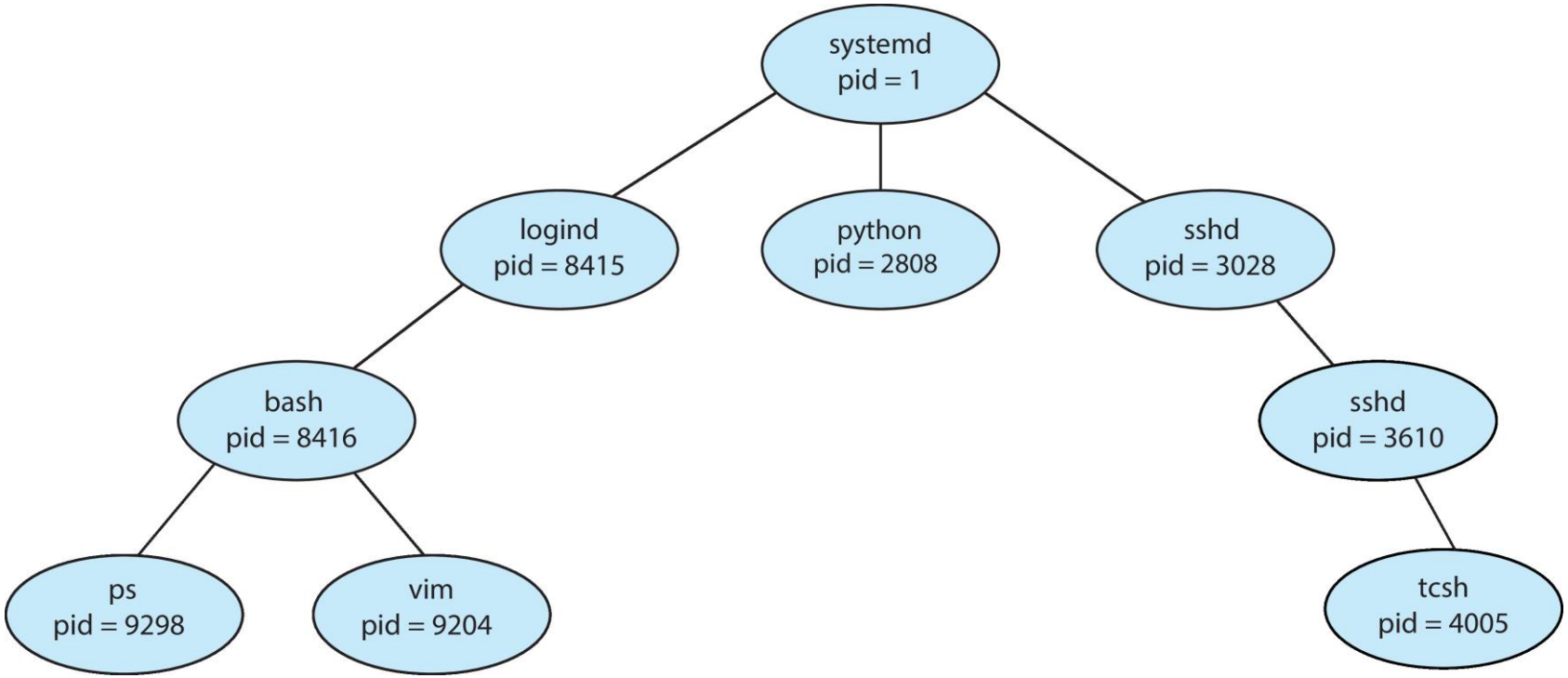
- Parent process create children processes, which, in turn create other processes, forming a tree of processes
- Generally, process identified and managed via a process identifier (pid)
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate

Process Creation (Cont.)

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - **`fork()`** system call creates new process
  - **`exec()`** system call used after a **`fork()`** to replace the process' memory space with a new program
  - Parent process calls **`wait()`** waiting for the child to terminate



A Tree of Processes in Linux



Process Termination

- Process executes last statement and then asks the operating system to delete it using the **`exit()`** system call.
  - Returns status data from child to parent (via **`wait()`**)
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **`abort()`** system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting, and the operating systems does not allow a child to continue if its parent terminates

Process Termination

- Some operating systems do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc., are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the `wait()` system call . The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```

- If no parent waiting (did not invoke `wait()`) process is a zombie
- If parent terminated without invoking `wait()`, process is an orphan

Android Process Importance Hierarchy

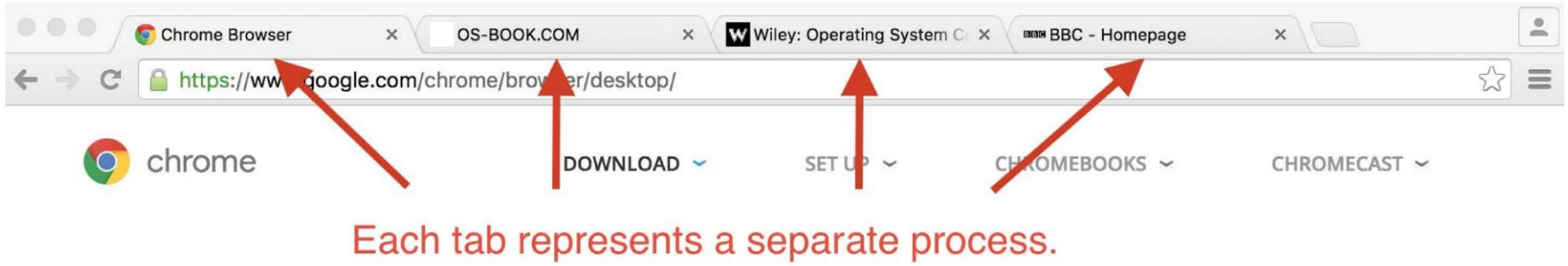
- Mobile operating systems often have to terminate processes to reclaim system resources such as memory. From **most** to **least** important:
  - Foreground process
  - Visible process
  - Service process
  - Background process
  - Empty process
- Android will begin terminating processes that are least important.

Multitasking in Mobile Systems

- Some mobile systems (e.g., early version of iOS) allow only one process to run, others suspended
- Due to screen real estate, user interface limits iOS provides for a
  - Single foreground process- controlled via user interface
  - Multiple background processes– in memory, running, but not on the display, and with limits
  - Limits include single, short task, receiving notification of events, specific long-running tasks like audio playback
- Android runs foreground and background, with fewer limits
  - Background process uses a service to perform tasks
  - Service can keep running even if background process is suspended
  - Service has no user interface, small memory use

Multiprocess Architecture – Chrome Browser

- Many web browsers ran as single process (some still do)
  - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 different types of processes:
  - Browser process manages user interface, disk and network I/O
  - Renderer process renders web pages, deals with HTML, Javascript. A new renderer created for each website opened
    - Runs in sandbox restricting disk and network I/O, minimizing effect of security exploits
  - Plug-in process for each type of plug-in



Interrupts in Operating Systems

What is an Interrupt?

- An **interrupt** is a signal sent to the CPU by hardware or software indicating an event that needs immediate attention.
- It **interrupts** the normal flow of execution so the OS can handle the event.
- Helps in **efficient CPU utilization** and **event-driven processing**.

Types of Interrupts:

Type	Description	Example
Hardware Interrupt	Generated by hardware devices	Keyboard input, Mouse click
Software Interrupt	Generated by programs via system calls or errors	int 0x80 in Linux, divide by zero
Timer Interrupt	Generated by the timer to support multitasking	CPU scheduling intervals
I/O Interrupt	Signals from I/O devices when ready	Disk I/O complete

Interrupt Handling Process

- **Steps:**
  - CPU receives an interrupt signal.
  - Saves the current state (Program Counter, Registers).
  - Transfers control to **Interrupt Service Routine (ISR)**.
  - ISR processes the event.
  - Restores saved state and resumes execution.

◆ Common Interrupts in OS

Interrupt	Purpose
Keyboard	Key press detection
Mouse	Mouse movement/click
Disk	Read/write complete
System Call	Request OS service from program
Timer	Time slice ended, context switch



START THE QUIZ

What is a system call?

A) A function used to call user-defined routines

C) A call to execute a loop

B) A call to execute a user program directly

D) A mechanism to request service from the operating system

Next

Which of the following is a system call used for process creation?

A) exit()

C) open()

B) printf()

D) fork()

Next

What happens when a system call is executed?

A) The program is terminated

C) The program sleeps

B) The program is terminated

D) The OS switches to kernel mode

Next

Which hardware device commonly generates interrupts?

A) Keyboard

C) ALU

B) RAM

D) CPU

Next

After a successful fork(), how many processes are created?

A) 2

C) 0

B) 1

D) Depends on the OS

Next

What is the role of an Interrupt Service Routine (ISR)?

A) To generate interrupts

C) To handle the event caused by the interrupt

B) To schedule processes

D) To stop the CPU

Well done!

Thanks